

Nilay - 21101255

# Implementation 1

```
def fibonacci_1(n):
```

```
    if n < 0: _____  $O(1)$   
        print("Invalid input!")
```

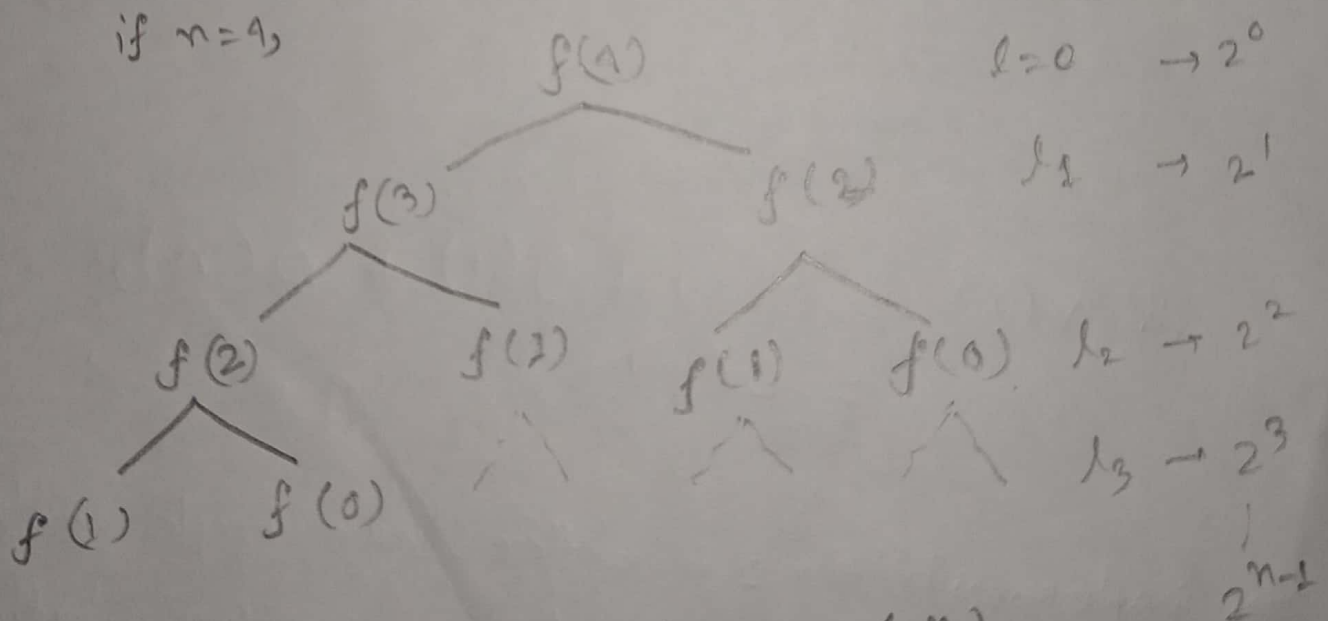
```
    elif n <= 1:  
        return n _____  $O(1)$ 
```

```
    else:  
        return fibonacci_1(n-1)+fibonacci_1(n-2) _____  $O(2^n)$ 
```

```
n = int(input("Enter a number: "))
```

```
nth_fib = fibonacci_1(n)
```

```
print("The {} th fibonacci number is {}".format(n,  
nth_fib))
```



$$\begin{aligned}\therefore T(n) &= O(1) + O(1) + O(2^n) \\ &= O(2^n)\end{aligned}$$

```
# Implementation 2
def fibonacci_2(n):
```

```
    if n < 0:
        return "Invalid Input"             $O(1)$ 
```

```
    if n <= 1:
        return n                                   $O(1)$ 
```

```
    fib = [0] * (n+1)
    fib[0] = 0
    fib[1] = 1
```

```
    for i in range(2, n+1):                           $O(n)$  }  $O(n)$ 
        fib[i] = fib[i-1] + fib[i-2]                   $O(n-1)$ 
```

```
    return fib[n]
```

```
n = int(input("Enter a number: "))
nth_fib = fibonacci_2(n)
print("The {} th fibonacci number is {}".format(n,
nth_fib))
```

$$\therefore T(n) = O(1) + O(1) + O(n) \\ = O(n)$$

So, implementation 2 is better than implementation 1  
as the  $T(n)$  is  $O(n)$  in implementation 2.

Ⓐ

$$\therefore T(n) = O(2^n)$$